UNIVERSIDAD DE MANILA

COLLEGE OF COMPUTING STUDIES

FINAL DOCUMENTATION

In partial fulfillment of the requirements for the subject

Human Computer Interaction - ITE 323

SUBMITTED BY (GROUP MEMBERS):

- **Doroja, Paisley Isha V. (Role 4: Planning & UX Lead)**
- **Latoja, Jazmin Eunice C. (Role 1: CSS, JS & HTML Developer)**
- **Tan, Carl Adrian A. (Role 2: System Framework Developer)**
- **Villanueva, Gwyneth Vienne G. (Role 3: Database & Queries Developer)**

SUBMITTED TO:

Prof. Mary Grace Nulud

ACADEMIC TERM / DATE:

October 9, 2026 / Academic Year 2026-2027

bikko.studio (studioNeo / sankobite-v3)

An Interactive Studio Independent Website and Secure Digital Asset Sharing Platform

Project Scope and Member Assignment Index

Group Member	Role Assignment	Core Subsystems	Documentation Sections Owned
Jazmin Eunice C. Latoja (Jaz)	Role 1: CSS, JS & HTML Developer	Semantic HTML5 landmarks, Liquid Glass Dynamic Island, 4-state morphing navigation, 60fps sub-2KB SVG vector engine, and canvas multi-format exporter	Section 2.3.3 (WCAG 2.1 AA Compliance), Section 3.1 (Wireframes 3.1.1 - 3.1.5), Sections 3.3.1 - 3.3.4 (System Prototypes)
Carl Adrian A. Tan (Adrian)	Role 2: System Framework Developer	Next.js 16 App Router, React 19 Server Components, WebAuthn Level 3 biometric passkeys, sub-250ms sign-in, RFC 6238 TOTP, 10-minute email OTPs, secure cookies, and PWA Service Worker caching	Section 2.3.1 (FR1, FR2, FR3, FR8), Section 2.3.2 (NFR1 - NFR6), Section 2.3.4 (Security Overview), Section 3.3.5 (PWA Offline Support)
Gwyneth Vienne G. Villanueva (Gwyneth)	Role 3: Database & Queries Developer	Azure Database for MySQL Flexible Server, 100% BCNF schema, connection pooling, WebAuthn credential tables, paginated feed/story queries, category filter indexing, and 15-minute temporary Azure Blob links	Section 1.2.2 (Specific Objective 2), Section 2.2.3 (Asset Integrity), Section 2.3.1 (FR4, FR5, FR6, FR7), Section 3.2.3 (Storyboard Scenario 3)
Paisley Isha V. Doroja (Paisley)	Role 4: Planning, Research & QA Lead	Project Proposal & Objectives, 4 core bottlenecks, 5 target user groups, behavioral personas (Erika, Aria, Gabriel), comparison matrix, 4 interactive storyboards, artist seeding (@tamimoon, @rella, etc.), and NVDA accessibility testing	Section 1 (Project Proposal), Section 2.1 & 2.2 (User Research & Needs), Section 3.2 (Storyboards 3.2.1, 3.2.2, 3.2.4), Section 4 (References)

Cross-Role Integration and Development Handoffs

1. Passwordless Sign-in: Role 1 (Modal UI) → Role 2 (WebAuthn biometric challenge) → Role 3 (MySQL public key verification)
2. Curated Feed & Stories: Role 4 (Content seeding & sub-2KB SVGs) → Role 3 (Paginated SQL queries) → Role 1 (Glass cards & double-tap like burst)
3. Protected Asset Export: Role 1 (Browser canvas rasterization) → Role 3 (Entitlement check & 15-min Azure SAS link) → Role 2 (TLS transport check)
4. Accessibility & Offline: Role 1 (HTML5 landmarks & focus rings) → Role 2 (PWA app shell cache) → Role 4 (NVDA screen reader testing)

Jazmin Eunice C. Latoja

Role 1: CSS, JS, and HTML (Frontend UI, Vector Graphics & Accessibility Developer)

Technical Documentation and Member Assignment: Jazmin Eunice C. Latoja

- **Member Name**: Jazmin Eunice C. Latoja (Jaz)
- **Assigned Role**: Role 1: CSS, JS, and HTML (Frontend UI, Vector Graphics and Accessibility Developer)
- **Project Title**: bikko.studio (System Architecture: studioNeo / sankobite-v3)
- **Institution**: Universidad de Manila, College of Computing Studies
- **Course**: Human Computer Interaction (ITE 323)

1. Role Overview and Scope of Work

Jaz serves as the Frontend UI, Vector Graphics, and Accessibility Developer. Her scope of work focuses on the client-side presentation layer, accessible HTML5 structure, responsive CSS layout systems, interactive vector graphics animations, and multi-format client asset exports.

Her primary goal is to deliver a smooth, visually appealing, and highly accessible user interface that runs efficiently across mobile phones, tablets, and desktop computers while maintaining full compliance with Web Content Accessibility Guidelines (WCAG 2.1 Level AA).

2. Core Technical Responsibilities

2.1. HTML5 Structure and Assistive Markup

1. **Semantic Page Layout (WCAG 2.1 Rule 4.1.2 - Name, Role, Value)**:
2. Build all page views using native HTML5 landmark elements: `<header>`, `<main>`, `<nav>`, `<aside>`, `<section>`, and `<footer>`.
3. Apply explicit accessibility roles (`role="navigation"`, `role="region"`, `role="feed"`, `role="dialog"`) and descriptive labels (`aria-label`, `aria-labelledby`) so assistive technologies can identify the purpose of every layout section.
4. Key files: `src/app/layout.tsx`, `src/components/feed/InstagramFeed.tsx`, `src/app/gallery/GalleryClient.tsx`.
5. **Skip Navigation Implementation (WCAG 2.1 Rule 2.4.1 - Bypass Blocks)**:
6. Implement a hidden "Skip to Main Content" button at the very beginning of the page body.
7. The button remains invisible to mouse users during regular browsing. When a keyboard user presses the Tab key for the first time on page load, the button becomes visible at the top left of the screen with a distinct focus ring.
8. Pressing Enter immediately shifts keyboard focus past repeated navigation links directly to `<main id="main-content">`.

9. Styling: `css .sn-skip-link { position: absolute; top: -100px; left: 1rem; background: #6366f1; color: #ffffff; padding: 0.75rem 1.25rem; border-radius: 9999px; font-weight: 600; z-index: 9999; transition: top 0.2s ease; } .sn-skip-link:focus { top: 1rem; outline: 3px solid #ffffff; outline-offset: 2px; }`
10. **Logical Focus and Tab Order (WCAG 2.1 Rules 2.1.1 and 2.4.7 - Keyboard Navigable and Focus Visible)**:
11. Enforce a predictable, top-to-bottom and left-to-right keyboard Tab sequence across the navigation island, story reels, feed cards, and popup dialogs.
12. Provide high-contrast focus outlines across all interactive elements (`outline: 2px solid #a855f7; outline-offset: 3px;`).
13. Implement focus trapping in popup windows (Sign-in Gateway, Asset Export Modal, Fullscreen Story Viewer) so keyboard navigation stays inside the open dialog until dismissed.
14. **Non-Text Alternatives (WCAG 2.1 Rule 1.1.1 - Non-text Content)**:
15. Provide descriptive text alternatives for all graphical elements.
16. Add meaningful `alt` attributes to raster images (such as artist profile avatars and story thumbnails).
17. Embed `<title>` and `<desc>` elements within native SVG vectors.
18. Provide hidden screen reader labels (`aria-label` or `<span className="sr-only">`) on icon-only buttons like the close icon, heart like button, bookmark button, and theme switcher.

2.2. CSS Styling, Design Tokens and Responsive Layouts

1. **Liquid Glass Dynamic Island (`.sn-nav-island`)**:
2. Build the floating navigation bar with modern glass styling: `css .sn-nav-island { position: fixed; top: 1rem; left: 50%; transform: translateX(-50%); backdrop-filter: blur(28px) saturate(190%) contrast(110%); -webkit-backdrop-filter: blur(28px) saturate(190%) contrast(110%); background: var(--sn-glass-surface, rgba(18, 18, 22, 0.65)); border: 1px solid var(--sn-glass-border, rgba(255, 255, 255, 0.12)); box-shadow: 0 20px 40px -15px rgba(0, 0, 0, 0.5); border-radius: 9999px; transition: all 0.4s ease; z-index: 900; }`
3. Key files: `src/app/nav.css` , `src/app/components/StudioNav.tsx` .
4. **Four-State Adaptive Navigation System**:
5. **State A (Expanded Desktop Top Bar)**: Idle state at the top of the page with full horizontal navigation links, brand logo, dashboard link, and theme controls.
6. **State B (Detached Split Pills)**: Triggers when the user scrolls down past 40 pixels. The navigation bar shrinks padding and splits into two separate floating pills (brand and main links on the left, passkey and theme controls on the right), keeping 92% of the vertical screen height completely clear for viewing artwork.
7. **State C (Compact Mobile Bottom 4-Tab Dock)**: On mobile viewports under 768px, anchors a clean 4-tab bar to the bottom of the screen (Feed, Projects, Vault, Profile/Login) for easy thumb navigation.
8. **State D (Full-Screen Overlay Menu)**: Triggers when the hamburger menu button is pressed, expanding a full-screen blurred glass backdrop with accessible navigation links and signed-in user status.
9. **Multi-Theme Design Tokens**:
10. Define CSS custom variables across three complete color modes:
    - **Light Theme**: Contrast ratio of 14.2:1 (exceeds WCAG 4.5:1 standard). `css :root { --sn-bg-primary: #f8f9fa; --sn-text-primary: #121316; --sn-glass-surface: rgba(255, 255, 255, 0.75); --sn-glass-border: rgba(0, 0, 0, 0.08); }`

- **Dark Theme**: Contrast ratio of 15.8:1 (exceeds WCAG 4.5:1 standard). `css [data-theme='dark'] { --sn-bg-primary: #0d0e12; --sn-text-primary: #f1f3f9; --sn-glass-surface: rgba(18, 20, 26, 0.70); --sn-glass-border: rgba(255, 255, 255, 0.12); }`
- **OLED High-Contrast Theme**: Contrast ratio of 21:1 (maximum digital display contrast). `css [data-theme='oled'] { --sn-bg-primary: #000000; --sn-text-primary: #ffffff; --sn-glass-surface: rgba(0, 0, 0, 0.95); --sn-glass-border: rgba(255, 255, 255, 0.35); }`

11. Key file: `src/app/globals.css`.
12. **Cross-Device Responsive Breakpoints**:
13. **Mobile Viewports (<768px)**: Single-column feed layout with full-width post cards, horizontal swipeable Story avatar carousel, and fixed bottom 4-tab dock.
14. **Tablet Viewports (768px to 1023px)**: Centered 640px post feed stream with floating Dynamic Island navigation bar.
15. **Desktop Viewports (>=1024px)**: Asymmetric two-column grid layout consisting of a fixed 320px left profile sidebar and a centered 640px feed stream.
16. **Core Wireframe Views Constructed**:
17. Wireframe 3.1.1: Studio Landing Workshop and Main Moving Stage
18. Wireframe 3.1.2: Dual-Mode Adaptive Dynamic Island Navigation System
19. Wireframe 3.1.3: Authenticated Instagram-Style Feed and Desktop Profile Sidebar
20. Wireframe 3.1.4: Interactive Asset Vault and Multi-Format Export Popup Modal
21. Wireframe 3.1.5: Passwordless WebAuthn / Biometric Sign-in Window

---

2.3. Client-Side JavaScript and Moving Vector Engine

1. **Code-Driven SVG Animation Engine**:
2. Implement GPU-accelerated mathematical vector animations running smoothly at 60 frames per second using under 2KB of file data per graphic.
3. Uses SVG bezier curve paths, CSS hardware layer acceleration (`transform: translate3d(0,0,0)`, `will-change: transform`), and `requestAnimationFrame` loops.
4. Completely replaces heavy 15MB MP4/WebM video loops to save mobile data and system memory.
5. Key files: `src/app/LandingClient.tsx`, `src/app/gallery/GalleryClient.tsx`, `src/app/components/BikkoMark.tsx`.
6. **CPU Saver and Motion Comfort Controls**:
7. **Page Visibility API**: Detects when the user switches browser tabs or minimizes the window (`document.hidden`), immediately halting animation loops and lowering background CPU usage to 0%. `typescript // Check if tab is inactive to preserve CPU and battery if (typeof document !== 'undefined' && document.hidden) { return; }`
8. **Motion Comfort Detection (WCAG 2.1 Rule 2.2.2)**: Automatically inspects the operating system setting `prefers-reduced-motion: reduce`. When active, moving animations stop automatically and display a crisp static vector fallback.
9. **Feed Micro-Interactions**:
10. Fullscreen Story progress bar that fills over 5.0 seconds with smooth linear progression and pauses on press-and-hold.
11. Glowing purple gradient avatar rings indicating unviewed artist stories.

12. Double-tap floating heart burst animation with dynamic particle expansion and smooth fade-out.
13. Fast touch feedback under 100 milliseconds for all clicks, tabs, and filter selections.
14. Key files: `src/components/feed/InstagramFeed.tsx`, `src/components/feed/StoryViewer.tsx`.
15. **In-Browser Multi-Format Exporter**:
16. Build client-side canvas rasterization engine (`HTMLCanvasElement`, `canvas.getContext('2d')`).
17. Reads raw mathematical SVG code from browser memory at 3840 x 2160 resolution in the Display P3 wide color gamut.
18. Converts the artwork directly on the client machine into downloadable SVG, PNG, JPEG, and animated GIF formats without requiring expensive server image rendering.
19. Key files: `src/app/gallery/GalleryClient.tsx`, `src/app/collection/CollectionClient.tsx`.

---

3. Documentation Sections Owned

Jaz is directly responsible for drafting, updating, and maintaining the following formal sections in the project report:

1. **Section 2.3.3: Universal Accessibility Standards Compliance (WCAG 2.1 AA)**:
2. Rule 1.1.1: Non-text Content (Image alt text, SVG title and description tags, hidden icon labels).
3. Rule 1.4.3: Contrast Minimum (Light mode 14.2:1, Dark mode 15.8:1, OLED mode 21:1).
4. Rule 2.1.1: Keyboard Navigable (All functions operable via keyboard).
5. Rule 2.2.2: Pause, Stop, Hide (Reduced motion detection and pause controls).
6. Rule 2.4.1: Bypass Blocks (Skip to Main Content button).
7. Rule 2.4.7: Focus Visible (High-contrast focus outlines on all interactive elements).
8. Rule 4.1.2: Name, Role, Value (Semantic HTML5 landmarks and ARIA roles).
9. **Section 3.1: Wireframes (3.1.1 to 3.1.5)**:
10. 3.1.1 Studio Landing Workshop and Main Moving Stage (Desktop and Mobile wireframe layouts).
11. 3.1.2 Dual-Mode Adaptive Dynamic Island Navigation System (States A, B, C, and D).
12. 3.1.3 Authenticated Instagram-Style Feed and Desktop Profile Sidebar (Two-column desktop grid and single-column mobile stream).
13. 3.1.4 Interactive Asset Vault and Multi-Format Export Window (Grid layout and export popup modal).
14. 3.1.5 Passwordless WebAuthn / Biometric Sign-in Window (Modal dialog with biometric and fallback options).
15. **Sections 3.3.1 to 3.3.4: System Prototypes**:
16. 3.3.1 Custom Liquid Glass Visual Engine (CSS backdrop filters, specular highlights, border radius).
17. 3.3.2 Responsive Feed Interface and Frosted Glass Style (Card layouts, story avatar rows, double-tap hearts).
18. 3.3.3 Moving Vector Animation and Multi-Format Pipeline (Mathematical SVG rendering, canvas export pipeline).
19. 3.3.4 Flexible Design System (Light, Dark, and OLED High-Contrast themes and CSS variables).

---

4. Cross-Role Integration and Development Handoffs

- **Handoff to Role 2 (Adrian - System Framework)**:

- Jaz builds the Sign-in Gateway modal dialog UI with accessible focus trapping. Adrian connects the WebAuthn challenge generation, passkey verification, and secure cookie endpoints.
- Jaz creates the offline user interface notification banner. Adrian connects the background Service Worker and network status listeners.
- **Handoff to Role 3 (Gwyneth - Database and Queries)**:
- Jaz designs the multi-format export popup dialog. Gwyneth validates user membership status in MySQL and creates the 15-minute temporary Azure Blob download URL.
- Jaz renders the continuous feed cards and category filter tabs. Gwyneth writes the paginated SQL queries and category indexes.
- **Handoff to Role 4 (Paisley - Research and QA)**:
- Paisley provides curated artist vector assets, color palettes, and descriptive alt text. Jaz integrates them into the responsive feed and asset vault components.
- Jaz submits completed UI views to Paisley for formal WCAG 2.1 Level AA testing using the NVDA screen reader, keyboard-only Tab sequences, and contrast analyzers.

Carl Adrian A. Tan

Role 2: System Framework (Backend Architecture, WebAuthn, Security & PWA Developer)

Technical Documentation and Member Assignment: Carl Adrian A. Tan

- **Member Name**: Carl Adrian A. Tan (Adrian)
- **Assigned Role**: Role 2: System Framework (Backend Architecture, WebAuthn, Security and PWA Developer)
- **Project Title**: bikko.studio (System Architecture: studioNeo / sankobite-v3)
- **Institution**: Universidad de Manila, College of Computing Studies
- **Course**: Human Computer Interaction (ITE 323)

1. Role Overview and Scope of Work

Adrian serves as the Backend Architecture, WebAuthn, Security, and Progressive Web Application (PWA) Developer. His scope of work encompasses the core Next.js 16 server framework, hardware-bound passwordless authentication (FIDO2 / WebAuthn Level 3), zero-knowledge fallback login pipelines, single sign-on (SSO) integrations, security middleware, and offline service worker caching engines.

His primary objective is to deliver a rock-solid, zero-password cloud backend architecture that authenticates users in under 250 milliseconds, defends against credential theft and timing attacks, and ensures offline system resilience.

2. Core Technical Responsibilities

2.1. Core Framework and Speed Optimization

1. **Next.js 16 App Router Architecture (NFR6 - Clean Code Architecture)**:
2. Architect the entire application using strict TypeScript and React 19 Server Components within the Next.js 16 App Router framework.
3. Enforce a clean architectural separation between server-rendered layouts and client interactive widgets:
    - Server Components handle database queries, authentication verification, and initial HTML generation on the server.
    - Client Components (annotated with `"use client"`) handle local state, touch gestures, and canvas animations.
4. Core files: `src/app/layout.tsx`, `src/app/page.tsx`, `src/app/api/auth/*`.
5. **Performance and Load Budgets (NFR1 - Speed and Performance Efficiency)**:
6. Optimize page delivery pipelines so First Contentful Paint (FCP) and Largest Contentful Paint (LCP) complete within 1.2 seconds on broadband internet and within 2.2 seconds on standard 4G mobile networks.
7. Minimize client-side bundle sizes by avoiding heavy third-party authentication client libraries, relying on native browser WebAuthn APIs and lightweight server actions.

8. Maintain sub-100 millisecond response times for user clicks and touch events through asynchronous background processing.

2.2. Passwordless FIDO2 / WebAuthn and Backup Authentication

1. **Biometric Passkey Flow (FR1 - Passkey Registration and Biometric Verification)**:
2. Implement W3C WebAuthn Level 3 cryptographic standards using `@simplewebauthn/server` and `@simplewebauthn/browser`.
3. **Challenge Generation**: The server creates high-entropy cryptographic challenges using Node.js `crypto.randomBytes(32)` with a 60-second expiration window to prevent replay attacks.
4. **Hardware Signature Verification**: Verify public key credentials signed by device hardware chips (Apple Secure Enclave, Android StrongBox, Windows Hello TPM, or external YubiKey hardware tokens).
5. Supports both user-verifying platform authenticators (Face ID, Touch ID, Windows Hello) and roaming security keys.
6. Core files: `src/lib/webauthn.ts`, `src/app/api/auth/passkey/login-options/route.ts`, `src/app/api/auth/passkey/login-verify/route.ts`, `src/app/api/auth/passkey/register-options/route.ts`, `src/app/api/auth/passkey/register-verify/route.ts`.
7. **Sub-250ms Sign-in Response**:
8. Streamline cryptographic challenge generation, signature validation, and database key retrieval to transition the user to their authenticated feed in under 250 milliseconds (benchmarked at approximately 180 milliseconds under normal network conditions).
9. **Password-Free Backup Sign-in (FR2 - Password-Free Backup Sign-in)**:
10. **6-Digit Email Access Code**: For users signing in from a new or unlinked device, generate a cryptographically random 6-digit numeric code valid for exactly 10 minutes. The plain code is dispatched via email while its secure SHA-256 hash is recorded in the database.
11. **RFC 6238 TOTP Authenticator Support**: Implement Time-Based One-Time Password verification compatible with Google Authenticator, Microsoft Authenticator, and 1Password, using 30-second time steps and drift tolerance.
12. **Hashed Single-Use Recovery Keys**: Provide users with single-use emergency backup recovery keys stored as one-way cryptographic hashes.
13. Core files: `src/lib/auth.ts`, `src/lib/totp.ts`, `src/lib/email.ts`, `src/app/api/auth/login/route.ts`, `src/app/api/auth/verify-otp/route.ts`.
14. **Social Single Sign-On (FR3 - Social Media Single Sign-On)**:
15. Configure federated OAuth 2.0 / OpenID Connect (OIDC) authentication supporting Google Identity Services and Microsoft Entra ID.
16. Enforce Proof Key for Code Exchange (PKCE) with code verifiers and SHA-256 code challenges, plus cryptographic state parameters to prevent Cross-Site Request Forgery (CSRF).
17. Automatically initialize a member profile and handle when a new user signs in for the first time.
18. Core files: `src/lib/oauth.ts`, `src/app/api/auth/oauth/[provider]/route.ts`, `src/app/api/auth/oauth/[provider]/callback/route.ts`.

2.3. Security Middleware and PWA Offline Engine

1. **Hardened Session Cookies (NFR2 - Security and Protection Standards)**:
2. Issue session tokens using encrypted HTTP cookies protected with industry-standard security flags:

- `HttpOnly`: Prevents client-side JavaScript from accessing session tokens, eliminating Cross-Site Scripting (XSS) token theft.
- `SameSite=Lax` / `SameSite=Strict`: Protects against Cross-Site Request Forgery (CSRF) by preventing browsers from attaching authentication cookies to unauthorized third-party requests.
- `Secure`: Restricts cookie transmission strictly to encrypted TLS connections.

3. Core file: `src/lib/auth.ts`.
4. **Timing-Attack Defense (NFR2 - Security and Protection Standards)**:
5. Use constant-time comparison algorithms (`crypto.timingSafeEqual`) whenever verifying OTP codes, digital signatures, session tokens, or API secrets.
6. Prevents attackers from using slight variations in response times to guess authentication secrets byte by byte.
7. Implementation example: ```typescript import crypto from "crypto";

   export function safeCompare(a: string, b: string): boolean { const bufA = Buffer.from(a, "utf-8"); const bufB = Buffer.from(b, "utf-8"); if (bufA.length !== bufB.length) { return false; } return crypto.timingSafeEqual(bufA, bufB); } ```
8. **Progressive Web App Service Worker Caching (FR8 - PWA Offline Support)**:
9. Register a background Service Worker caching core static assets for instant loading and home-screen installation:
   - Application layout shell (`/`)
   - Web fonts and icon glyphs
   - Vector SVG files and artwork data
   - Compiled CSS style sheets and JavaScript bundles
10. Core files: `public/sw.js`, `src/app/manifest.ts`, `src/app/components/ServiceWorkerRegister.tsx`.
11. **Offline Fallback Handler (NFR5 - Reliability and Offline Grace)**:
12. Detect lost internet connectivity using browser event listeners (`navigator.onLine`, `window.addEventListener('offline')`).
13. Intercept failed network requests in the Service Worker and serve cached pages from local storage.
14. Trigger a clean, friendly offline notification banner across the UI without application crashes or white screen freezes.

---

3. Documentation Sections Owned

Adrian is directly responsible for drafting, updating, and maintaining the following formal sections in the project report:

1. **Section 2.3.1: Functional Requirements (FR1, FR2, FR3, FR8)**:
2. FR1: Passkey Registration and Biometric Verification (WebAuthn Level 3, challenge generation, signature checks, sub-250ms latency).
3. FR2: Password-Free Backup Sign-in (10-minute 6-digit email OTPs, RFC 6238 TOTP authenticator verification, single-use recovery keys).
4. FR3: Social Media Single Sign-On (Google and Microsoft OAuth 2.0 with PKCE and auto profile creation).
5. FR8: Progressive Web App Offline Support (Service worker caching, home-screen manifest, offline app shell).
6. **Section 2.3.2: Non-Functional Requirements (NFR1 to NFR6)**:

7. NFR1: Speed and Performance Efficiency (1.2s fast connection, 2.2s 4G mobile, sub-100ms click latency, 0% idle CPU).
8. NFR2: Security and Protection Standards (HttpOnly SameSite Secure cookies, constant-time byte comparisons, TLS encryption).
9. NFR3: Universal Accessibility (System support for WCAG AA standards and high contrast).
10. NFR4: Cross-Device Responsiveness (Mobile, tablet, desktop, and 4K display support).
11. NFR5: Reliability and Offline Grace (Offline network fallback banner, graceful error handling).
12. NFR6: Clean Code Architecture (Strict TypeScript, React 19 Server Components, Next.js 16 App Router).
13. **Section 2.3.4: Security Protection Overview (Multi-Layer Security Defense)**:
14. Document the multi-layer security architecture:
    - Layer 1: Hardware Sign-in via WebAuthn Biometrics and Roaming Security Keys.
    - Layer 2: Data Transport via TLS Encrypted Connections.
    - Layer 3: Session Safety via HttpOnly, SameSite, and Secure Cookies.
    - Layer 4: Timing-Attack Defense using equal-time byte checks.
    - Layer 5: Cloud File Rules using 15-minute temporary read-only download links.
15. **Section 3.3.5: Progressive Web Application (PWA) Offline Support and Memory System**:
16. Document the Service Worker cache architecture, cache-first routing rules, offline asset fallback flows, and device memory management.

---

4. Cross-Role Integration and Development Handoffs

- **Handoff to Role 1 (Jaz - Frontend UI)**:
- Adrian supplies the authentication endpoints (`/api/auth/passkey/login-options` , `/api/auth/passkey/login-verify` , `/api/auth/login` , `/api/auth/verify-otp`) to Jaz's Sign-in Gateway modal dialog.
- Adrian creates the Service Worker registration and network offline broadcast events that toggle Jaz's offline UI banner.
- **Handoff to Role 3 (Gwyneth - Database and Queries)**:
- Adrian passes verified user credentials, public keys, authenticator registration data, and OTP hashes to Gwyneth's MySQL tables (`webauthn_credentials` , `auth_magic_tokens` , `totp_credentials` , `totp_backup_codes`).
- Adrian uses Gwyneth's subscriber entitlement queries to authorize user download requests before issuing signed cookie sessions.
- **Handoff to Role 4 (Paisley - Research and QA)**:
- Adrian provides working passkey and OTP authentication workflows to Paisley for end-to-end security verification and biometric sign-in testing.
- Paisley benchmarks Adrian's sign-in response times against the sub-250 millisecond performance requirement and tests offline recovery behaviors.

Gwyneth Vienne G. Villanueva
Role 3: Database and Queries (Cloud Database, Schema & Secure Storage Developer)

Technical Documentation and Member Assignment: Gwyneth Vienne G. Villanueva

- **Member Name**: Gwyneth Vienne G. Villanueva (Gwyneth)
- **Assigned Role**: Role 3: Database and Queries (Cloud Database, Schema and Secure Storage Developer)
- **Project Title**: bikko.studio (System Architecture: studioNeo / sankobite-v3)
- **Institution**: Universidad de Manila, College of Computing Studies
- **Course**: Human Computer Interaction (ITE 323)

1. Role Overview and Scope of Work

Gwyneth serves as the Cloud Database, Schema, and Secure Storage Developer. Her scope of work encompasses relational database modeling (100% Boyce-Codd Normal Form compliance), Azure Database for MySQL Flexible Server provisioning, high-performance connection pooling, security credential storage, continuous feed and category query optimization, and time-gated Azure Blob Storage integration.

Her primary objective is to safeguard artist assets against unauthorized copying, hotlinking, and scraping while delivering instant, index-backed query responses for the social art feed and digital asset vault.

2. Core Technical Responsibilities

2.1. Cloud Database Setup and User Schemas

1. **Azure MySQL Flexible Server Provisioning (Specific Objective 2, Section 2.3.4)**:
2. Configure the managed Azure Database for MySQL Flexible Server using `utf8mb4` character set and `utf8mb4_unicode_ci` collation to support international creator handles and emoji reactions.
3. Enforce transport layer security using TLS 1.3 with official DigiCert Global Root CA certificates.
4. Implement high-performance connection pooling using `mysql2/promise` with configurable limits (`waitForConnections: true`, `connectionLimit: 10`, `queueLimit: 0`) to prevent connection exhaustion during traffic spikes.
5. Enforce parameterized queries using prepared statements (`?` placeholders) for 100% protection against SQL injection attacks.
6. Core files: `src/lib/db.ts`, `scripts/schema.sql`.
7. **Member Profile and Badge Schema (100% BCNF Compliant)**:
8. Design relational tables where every determinant is a superkey, eliminating data redundancy and update anomalies: `sql CREATE TABLE IF NOT EXISTS `accounts` ( `id` BIGINT UNSIGNED NOT NULL AUTO_INCREMENT, `email` VARCHAR(255) NOT NULL, `handle` VARCHAR(64) NOT NULL, `display_name` VARCHAR(255) NOT NULL, `avatar_url` TEXT NULL DEFAULT NULL, `bio` TEXT NULL DEFAULT NULL,`

`website_url` VARCHAR(512) NULL DEFAULT NULL, `is_verified` TINYINT(1) NOT NULL DEFAULT 0, `system_role` ENUM('user', 'curator', 'admin') NOT NULL DEFAULT 'user', `artist_distinction` ENUM('none', 'artist', 'featured', 'partner') NOT NULL DEFAULT 'none', `created_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP, `updated_at` TIMESTAMP NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP, PRIMARY KEY (`id`), UNIQUE KEY `idx_accounts_email` (`email`), UNIQUE KEY `idx_accounts_handle` (`handle`), KEY `idx_accounts_is_verified` (`is_verified`), KEY `idx_accounts_artist_distinction` (`artist_distinction`)) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4 COLLATE=utf8mb4_unicode_ci;

9. Stores user profiles, Member IDs (#1042), and verification distinctions (Partner, Featured Artist, Partner Verified).
10. **Authentication Credential Storage**:
11. **WebAuthn Credentials Table (`webauthn_credentials`)**:
    - Stores hardware credential IDs, public keys (`public_key` BLOB/TEXT), monotonic signature counters to detect authenticator cloning, device transport types (internal, usb, nfc, ble), and backup eligibility flags.
12. **Magic Tokens Table (`auth_magic_tokens`)**:
    - Stores 6-digit email OTP hashes (`token_hash` VARCHAR(64)), issuance timestamps, and expiration cutoffs (10-minute validity window).
13. **TOTP Credentials Table (`totp_credentials`)**:
    - Stores encrypted shared secrets (`secret_ciphertext`), confirmation status, and initialization vectors.
14. **Backup Codes Table (`totp_backup_codes`)**:
    - Stores salted hashes of single-use emergency recovery keys with consumption timestamps (`used_at`).
15. **OAuth Providers Table (`account_oauth_providers`)**:
    - Maps third-party Google and Microsoft identities to internal account records.

2.2. Feed, Story and Asset Vault Queries

1. **Continuous Feed and Active Story Queries (FR4 - Curated Art Feed)**:
2. Write paginated SQL queries joining `artworks` and `accounts` to stream creative posts with creator metadata (handle, avatar, location, timestamps, format tags): `sql SELECT a.id, a.code, a.title, a.description, a.category, a.like_count, a.comment_count, a.created_at, acc.handle, acc.display_name, acc.avatar_url, acc.artist_distinction FROM artworks a INNER JOIN accounts acc ON a.account_id = acc.id WHERE a.status = 'published' ORDER BY a.created_at DESC LIMIT ? OFFSET ?;`
3. Active 24-hour Story query retrieving recent artwork reels published within the last day: `sql SELECT * FROM artworks WHERE created_at >= NOW() - INTERVAL 24 HOUR ORDER BY created_at DESC;`
4. **Category Filter Indexing**:
5. Optimize feed filter tabs (All Works, Partners, and Featured) by indexing `artist_distinction` and `category` :
    - All Works: Queries all published posts (`WHERE status = 'published'`).
    - Partners: Filters by `acc.artist_distinction = 'partner'` .
    - Featured: Filters by `acc.artist_distinction = 'featured'` .

6. Return dynamic count headers for UI badge chips (`ALL WORKS (12)` , `PARTNERS (5)`).
7. **Engagement Metrics and Transactional Integrity**:
8. **Double-Tap Like Counter**: Use the `artwork_likes` table with a composite primary key `(account_id, artwork_id)` to enforce unique likes per user. Execute an atomic transaction to insert the like record and update the cached `like_count` in the `artworks` table: `sql START TRANSACTION; INSERT IGNORE INTO artwork_likes (account_id, artwork_id) VALUES (?, ?); UPDATE artworks SET like_count = (SELECT COUNT(*) FROM artwork_likes WHERE artwork_id = ?) WHERE id = ?; COMMIT;`
9. Track comment totals, user bookmarks, and download counts (`artwork_downloads` table) for artist analytics.
10. **Asset Vault Catalog Queries (FR6 - Multi-Format Asset Storage Vault)**:
11. Structure database queries for the Digital Asset Vault filtering across All Categories, Vector Stems, Color Studies, and Typography: `sql SELECT aa.id, aa.asset_type, aa.file_size_bytes, aa.mime_type, a.title, a.code, acc.handle FROM artwork_assets aa INNER JOIN artworks a ON aa.artwork_id = a.id INNER JOIN accounts acc ON a.account_id = acc.id WHERE (? IS NULL OR a.category = ?) ORDER BY a.created_at DESC;`

2.3. Cloud Blob Storage and Time-Gated Download Links

1. **Membership Permission Checks (Specific Objective 2, FR7 - Time-Limited Secure Download Links)**:
2. Before any master vector or high-resolution file is provided, query the database to verify active subscription or verified partner standing: `sql SELECT id, status, expires_at FROM subscribers WHERE account_id = ? AND status = 'active' AND expires_at > NOW();`
3. Deny request if subscriber status is inactive, expired, or unverified.
4. **15-Minute Temporary Download Links (Shared Access Signatures)**:
5. Integrate Azure Blob Storage using `@azure/storage-blob` (`src/lib/azure-storage.ts`).
6. Generate secure Shared Access Signature (SAS) URLs valid for exactly 15 minutes (`AZURE_STORAGE_SAS_EXPIRY_MINUTES = 15`).
7. Apply strict read-only permissions (`BlobSASPermissions.parse("r")`).
8. Attach clean download headers to prompt immediate browser saving rather than direct viewing: `contentDisposition: 'attachment; filename="studioNeo-artwork-master.svg"'` .
9. Benefits:
    - Eliminates permanent public URLs.
    - Prevents automated bots and web crawlers from scraping high-resolution art.
    - Blocks hotlinking and unauthorized link forwarding across the web.

---

3. Documentation Sections Owned

Gwyneth is directly responsible for drafting, updating, and maintaining the following formal sections in the project report:

1. **Section 1.2.2: Specific Objective 2 (Secure Cloud File Sharing System)**:
2. Define the database-backed permission checking mechanism and encrypted cloud storage architecture designed to stop art theft and direct file link scraping.
3. **Section 2.2.3: Asset Integrity and Art Protection**:

4. Detail user needs surrounding creative asset ownership, protection against unauthorized AI scrapers, guaranteed file safety, and reliable downloads for paid members.
5. **Section 2.3.1: Functional Requirements (FR4, FR5, FR6, FR7)**:
6. FR4: Curated Art Feed (Paginated feeds, 24-hour Story reels, category filter indexes, like transactions).
7. FR5: Desktop Profile Sidebar (Database queries retrieving user statistics, member IDs, badges, and security status).
8. FR6: Multi-Format Asset Storage Vault (Catalog schema for Vector Stems, Color Studies, and Typography).
9. FR7: Time-Limited Secure Download Links (Entitlement validation, 15-minute temporary read-only SAS generation).
10. **Section 3.2.3: Storyboard Scenario 3 (Asset Discovery, Multi-Format Conversion and Time-Gated Download)**:
11. Document the step-by-step user interaction flow from gallery inspection to database permission verification, SAS token generation, and secure file download.

---

4. Cross-Role Integration and Development Handoffs

- **Handoff to Role 1 (Jaz - Frontend UI)**:
- Gwyneth delivers optimized SQL query results and JSON endpoints feeding Jaz's responsive feed cards, filter chips, and profile sidebar.
- When a member clicks download in Jaz's export popup modal, Gwyneth runs the entitlement check and generates the 15-minute temporary link.
- **Handoff to Role 2 (Adrian - System Framework)**:
- Gwyneth establishes the connection pool and provides database helper functions (`query()` , `getAccountById()` , `saveWebAuthnCredential()`) to Adrian's API routes.
- Adrian's authentication middleware validates session cookies and passes the authenticated `accountId` to Gwyneth's query functions.
- **Handoff to Role 4 (Paisley - Research and QA)**:
- Paisley provides artist seed data (profile bios, handles, badges, showcase artwork descriptions) for Gwyneth to populate into MySQL tables.
- Paisley tests the 15-minute expiration window to ensure download links expire on schedule and verify that unauthorized users are blocked.

Paisley Isha V. Doroja

Role 4: Inputting, Research, and Planning (UX Research, Content Seeding & QA Lead)

Technical Documentation and Member Assignment: Paisley Isha V. Doroja

- **Member Name**: Paisley Isha V. Doroja (Paisley)
- **Assigned Role**: Role 4: Inputting, Research, and Planning (UX Research, Content Seeding and QA Lead)
- **Project Title**: bikko.studio (System Architecture: studioNeo / sankobite-v3)
- **Institution**: Universidad de Manila, College of Computing Studies
- **Course**: Human Computer Interaction (ITE 323)

1. Role Overview and Scope of Work

Paisley serves as the UX Research, Content Seeding, and Quality Assurance (QA) Lead. Her scope of work encompasses project proposal formulation, target user group research, behavioral persona development, user pain point identification, interactive storyboard design, curated artwork and creator seeding, vector graphic optimization, universal accessibility auditing (WCAG 2.1 Level AA), and compilation of the formal HCI documentation manuscript.

Her primary objective is to align the technical architecture with genuine human-computer interaction needs, ensuring the platform is intuitive, accessible to users with disabilities, richly populated with high-quality creative content, and rigorously tested against academic and industry standards.

2. Core Technical Responsibilities

2.1. Project Scope, Objectives and User Research

1. **Proposal and Problem Definition (Section 1)**:
2. **Project Title (Section 1.1)**: bikko.studio (Full descriptive title: An Interactive Studio Independent Website and Secure Digital Asset Sharing Platform Using Passwordless WebAuthn Sign-in and Fast Computer Graphics).
3. **General Objective (Section 1.2.1)**: Design, build, test, and launch bikko.studio as a high-quality cloud web platform delivering interactive moving artwork and protected digital assets via passwordless security, dynamic cloud access checks, and full WCAG 2.1 Level AA accessibility.
4. **Five Specific Objectives (Section 1.2.2)**:
    1. Build a multi-step passwordless sign-in system using WebAuthn biometrics, 6-digit email OTPs, TOTP authenticators, and social logins.
    2. Create a secure cloud file sharing system with database permission checks and 15-minute temporary download links.
    3. Develop a moving image engine and file storage vault using lightweight animated SVGs under 2KB and in-browser canvas format conversion.

4. Design an adaptive, accessible user interface featuring the morphing Dynamic Island navigation bar and multi-theme tokens.
5. Measure system speed, performance, and security under heavy user loads.

5. **Five Target User Groups (Section 1.3)**:
    1. Digital Illustrators, Animators, and Visual Artists
    2. Creative Technologists and Web Developers
    3. Digital Asset Collectors and Subscribers
    4. Design Studio Directors and Commercial Curators
    5. Users Relying on Assistive Tools and Various Setup Conditions
6. **Four Core System Problems (Section 1.4)**:
    1. Password Fatigue and Account Hijacking (phishing, password reuse, reset loops).
    2. Asset Theft and Uncontrolled Direct Links (permanent URLs exposed to scraping and bandwidth theft).
    3. Slow Performance from Heavy Video Files (15MB MP4 loops causing lag and mobile data drain).
    4. Stiff Navigation and Accessibility Barriers (mobile screen obstruction, unannounced buttons, broken focus).
7. **User Personas and Behavioral Profiles (Section 2.1.1)**:
8. **Persona A: Erika Vance (Age: 27)**:
    - Role: Senior Illustrator and Freelance Concept Artist.
    - Tools: macOS Sonoma, 4K Display, iPad Pro with Apple Pencil, Safari and Chrome.
    - Mindset: Cares deeply about color fidelity and vector crispness. Constantly concerned about online art theft, direct link scraping, and unauthorized AI training.
    - Goals: Showcase moving artwork with zero blurriness, verify paying supporters, and distribute master vectors safely.
9. **Persona B: Aria De Leon (Age: 21)**:
    - Role: Design Student and Digital Art Collector.
    - Tools: iPhone 15 Pro, iOS 18, Mobile Safari, Campus Wi-Fi.
    - Mindset: Browses quickly on mobile, accustomed to modern touch habits (stories, double-taps, continuous scrolling).
    - Goals: Browse art feeds on mobile, enjoy 5-second animated story previews, and download wallpapers directly to cloud storage.
10. **Persona C: Gabriel Santos (Age: 32)**:
    - Role: Accessibility Consultant and Digital Curator.
    - Tools: Windows 11 with NVDA Screen Reader, High-Contrast Monitor, Keyboard-Only Navigation.
    - Mindset: Completely relies on logical HTML markup, keyboard shortcuts, and clear screen contrast. Experiences motion sickness from unconstrained animations.
    - Goals: Navigate art archives independently using clear keyboard commands and screen reader speech announcements.
11. **User Group Comparison Table (Section 2.1.2)**:
12. Systematically evaluate parameter variations across groups:
    - **Age Range**: Artists (20 to 40), Technologists (19 to 35), Assistive Users (22 to 55).
    - **Tech Skill**: Artists (Medium), Technologists (High / Expert), Assistive Users (High with Assistive Tools).

- **Main Device**: Artists (Desktop / Tablet 65%), Technologists (Multi-Screen Desktop 80%), Assistive Users (Desktop / Laptop 85%).
- **Sign-in Choice**: Artists (Touch ID / Social), Technologists (Hardware Key / Passkey), Assistive Users (Keyboard Passkey / Email Code).
- **Key Need**: Artists (Asset Safety and Quality), Technologists (Vector Downloads and Clean Code), Assistive Users (Accessibility and High Contrast).

13. **Needs and Requirements Translation (Section 2.2)**:
14. Map user pain points into structured specifications covering usability (zero-password relief, easy recovery), visual presentation (crisp SVGs, subtle touch feedback), asset integrity (temporary links, verified downloads), and accessibility (focus outlines, motion comfort, OLED true-black mode).

2.2. Journey Mapping, Content Seeding and Asset Preparation

1. **Step-by-Step Interactive Storyboards (Section 3.2)**:
2. **Scenario 1: Easy Biometric Passkey Setup and Sign-in (User: Marcus Chen)**:
   - Step 1 (Start): Marcus clicks the glowing "Sign In" pill on the navigation bar.
   - Step 2 (Challenge Request): Browser requests a cryptographic challenge from the server.
   - Step 3 (Biometric Prompt): Browser triggers native Touch ID on macOS.
   - Step 4 (Hardware Scan): Marcus touches the sensor; security enclave signs the challenge.
   - Step 5 (Verification): Server verifies the digital signature against the stored public key.
   - Step 6 (Success): Within 180 milliseconds, the app transitions smoothly to the active feed.
3. **Scenario 2: Creative Exploration and Story Reel Interaction (User: Aria De Leon)**:
   - Step 1 (Story Discovery): Aria opens the feed and sees a glowing purple ring around artist @tamimoon.
   - Step 2 (Story Window): Tapping opens fullscreen story mode with an animated vector and a 5-second progress bar.
   - Step 3 (Double-Tap Like): Double-tapping triggers a floating heart burst animation and increments like counts.
   - Step 4 (Filter Tabs): Aria exits and taps "Partners"; feed updates instantly.
   - Step 5 (Smart Docking): Scrolling down hides the bottom dock for unobstructed art viewing; scrolling up restores it.
4. **Scenario 3: Secure Asset Discovery, Multi-Format Conversion and Download (User: Erika Vance)**:
   - Step 1 (Gallery Inspect): Erika selects "Vector Geometry Picture 04".
   - Step 2 (Format Choice): In the export modal, she chooses "PNG (High-Quality Transparent Image)".
   - Step 3 (Browser Conversion): Client-side canvas rasterizes native vector code at 3840 x 2160 Display P3 resolution.
   - Step 4 (Member Check): Server confirms Erika's active paid membership.
   - Step 5 (Safe Link Generation): Server returns a 15-minute temporary read-only download link.
   - Step 6 (Direct Download): Browser downloads the file directly without exposing permanent storage paths.
5. **Scenario 4: Accessibility Navigation and High-Contrast Theming (User: Gabriel Santos)**:
   - Step 1 (Skip Link): Gabriel presses Tab on load; "Skip to Main Content" button appears with high-contrast ring.

- Step 2 (Theme Access): Tabbing brings focus to "Theme Settings", announced clearly by NVDA.
- Step 3 (OLED Mode): Gabriel opens the menu and selects "OLED High-Contrast" (21:1 contrast ratio, true black).
- Step 4 (Motion Protection): App detects Windows "Reduce Motion" setting and halts all background vector animations automatically.

6. **Creator Profile and Feed Content Seeding**:
7. Populate authentic artist accounts, avatars, bios, and badges:
    - `@tamimoon` (Tokyo, Japan; Partner Verified; Modern fashion and anime illustrative aesthetics).
    - `@rella` (Tokyo, Japan; Partner Verified; Symphony series, delicate light studies).
    - `@sankobite` (Lead Creator; Core architectural vectors, cyber-aesthetic studies).
    - `@yoneyamai` (Tokyo, Japan; Featured Artist; Kinetic looping posters).
    - `LAM` (Tokyo, Japan; Featured Artist; High-contrast street fashion graphics).
8. Key files populated: `src/tamimoonData.ts`, `src/rellaData.ts`, `src/yoneyamaiData.ts`, `src/assets/*`.
9. **Vector Asset Preparation and Vault Seeding**:
10. Author and optimize sub-2KB SVG animation files using clean math curves.
11. Prepare master 3840 x 2160 vectors with Display P3 color definitions.
12. Tag artwork with precise categories (Vector Stems, Color Studies, Typography) and embed detailed screen-reader descriptions into `<title>` and `<desc>` elements.

2.3. Accessibility Auditing, QA and Manuscript Compilation

1. **WCAG 2.1 Level AA Compliance Testing**:
2. Perform hands-on accessibility evaluations using the NVDA screen reader on Windows 11.
3. Test complete keyboard-only navigation workflows using Tab, Shift+Tab, Enter, Space, and Escape keys.
4. Verify color contrast ratios using digital color analyzers:
    - Light Theme: 14.2:1 (exceeds 4.5:1 requirement).
    - Dark Theme: 15.8:1 (exceeds 4.5:1 requirement).
    - OLED Theme: 21:1 (maximum digital contrast).
5. Verify that animations freeze when `prefers-reduced-motion` is active.
6. **HCI Final Documentation Manuscript Management**:
7. Compile, format, and organize the complete 60-page Human Computer Interaction (ITE 323) manuscript following Universidad de Manila College of Computing Studies standards.
8. Ensure consistent document headings, page numbers, ASCII wireframe diagrams, and member role matrices.
9. **Academic and Industry Reference Citations (Section 4)**:
10. Manage official citations:
    1. FIDO Alliance & World Wide Web Consortium (W3C). (2025). Web Authentication: An API for accessing Public Key Credentials Level 3 (WebAuthn Level 3). W3C Candidate Recommendation.
    2. World Wide Web Consortium (W3C). (2018). Web Content Accessibility Guidelines (WCAG) 2.1. W3C Recommendation.
    3. International Organization for Standardization (ISO). (2012). ISO/IEC 40500:2012 Information technology - W3C Web Content Accessibility Guidelines (WCAG) 2.0. Geneva: ISO.

4. Internet Engineering Task Force (IETF). (2011). TOTP: Time-Based One-Time Password Algorithm (RFC 6238). Request for Comments.
5. Open Web Application Security Project (OWASP). (2023). OWASP Application Security Verification Standard (ASVS) 4.0.3. OWASP Foundation.
6. Owens, K., et al. (2021). User Perceptions of the Usability and Security of Smartphones as FIDO2 Roaming Authenticators. SOUPS 2021.
7. Ali, M., et al. (2024). FIDO2 Password-less Authentication: Security-Usability Continuum. IEEE Access.
8. Microsoft Corporation. (2026). Azure Database for MySQL Flexible Server & Azure Blob Storage Documentation. Microsoft Learn.
9. Meta Open Source & Vercel. (2026). React 19 Server Components and Next.js 16 App Router Architecture. Vercel Documentation.

---

3. Documentation Sections Owned

Paisley is directly responsible for drafting, updating, and maintaining the following formal sections in the project report:

1. **Section 1: Project Proposal**:
2. 1.1 Project Title (System names and identifiers).
3. 1.2 Objectives (1.2.1 General Objective and 1.2.2 Specific Objectives 1 through 5).
4. 1.3 Target Users (Five designated user groups).
5. 1.4 Problem Statement (Four core system problems).
6. **Section 2.1 and 2.2: User Research and User Needs**:
7. 2.1.1 User Personas and Behavioral Profiles (Personas A, B, and C).
8. 2.1.2 Demographic and Technological Literacy Segmentation (Comparison Table).
9. 2.2 User Needs (Usability, visual presentation, asset integrity, and assistive device support).
10. **Section 3.2: Storyboards**:
11. 3.2.1 Scenario 1: Easy Biometric Passkey Setup and Sign-in (Marcus Chen).
12. 3.2.2 Scenario 2: Creative Exploration and Story Reel Interaction (Aria De Leon).
13. 3.2.4 Scenario 4: Accessibility Navigation and High-Contrast Theming (Gabriel Santos).
14. **Section 4: References**:
15. Compile and verify all 9 formal citations across W3C, ISO, IETF, OWASP, academic journals, and cloud documentation.

---

4. Cross-Role Integration and Development Handoffs

- **Handoff to Role 1 (Jaz - Frontend UI)**:
- Paisley provides optimized SVG files, artist descriptions, and color values for Jaz's responsive feed and story reels.
- Paisley conducts NVDA screen reader audits and keyboard Tab sequence tests on Jaz's wireframe views and Liquid Glass components.
- **Handoff to Role 2 (Adrian - System Framework)**:

- Paisley tests Adrian's biometric passkey registration, email OTP generation, and fallback flows against user experience benchmarks.
- Paisley verifies that offline Service Worker alerts match user expectations during network dropouts.
- **Handoff to Role 3 (Gwyneth - Database and Queries)**:
- Paisley delivers structured artist profile information and artwork metadata for Gwyneth to seed into the `accounts` and `artworks` database tables.
- Paisley conducts test downloads to confirm that 15-minute SAS links expire accurately and protect creator assets.